Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа

по курсу «Моделирование»

«Решение баллистической задачи»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Домрачева А.Б.

Москва 2026

1 Постановка задачи

Шар заданного радиуса бросается под углом к горизонту с заданной начальной скоростью. Используя математические модели, необходимо найти дальность полёта шара и построить траекторию. Начальные условия: угол бросания, начальная скорость, старт из точки $(0, 0)$. Дополнительные параметры: материал шара, радиус шара, коэффициент сопротивления воздуха.

Вариант 29: угол $\alpha = 53$, материал — железо, начальная скорость $v_0 = 185$ м/с, радиус шара $r = 0{,}29$ м.

2 Построение математической модели

Модель Галилея описывает полёт тела в пустоте под действием только силы тяжести. Сопротивление воздуха не учитывается, поэтому горизонтальная и вертикальная составляющие движения независимы: по горизонтали тело движется равномерно, по вертикали — с постоянным ускорением g. Параметры модели: v_0 — начальная скорость, α — угол к горизонту, g — ускорение свободного падения. Траектория получается параболой, дальность полёта выражается через v_0, α и g.

Модель Ньютона учитывает сопротивление воздуха, пропорциональное квадрату скорости. Коэффициент β задаёт величину силы сопротивления и связывает площадь поперечного сечения тела S, плотность воздуха $\rho_{\text{воз}}$ и безразмерный коэффициент лобового сопротивления C:

$$\beta = \frac{CS\rho_{\text{воз}}}{2}. \tag{1}$$

Масса снаряда m связана с радиусом шара r и плотностью материала ρ_m (для железа около 7850 кг/м^3). Система дифференциальных уравнений связывает производные координат (x, y) и компонент скорости (u, w) с силой тяжести и силой сопротивления.

Схема построения модели представлена на рисунке 1.

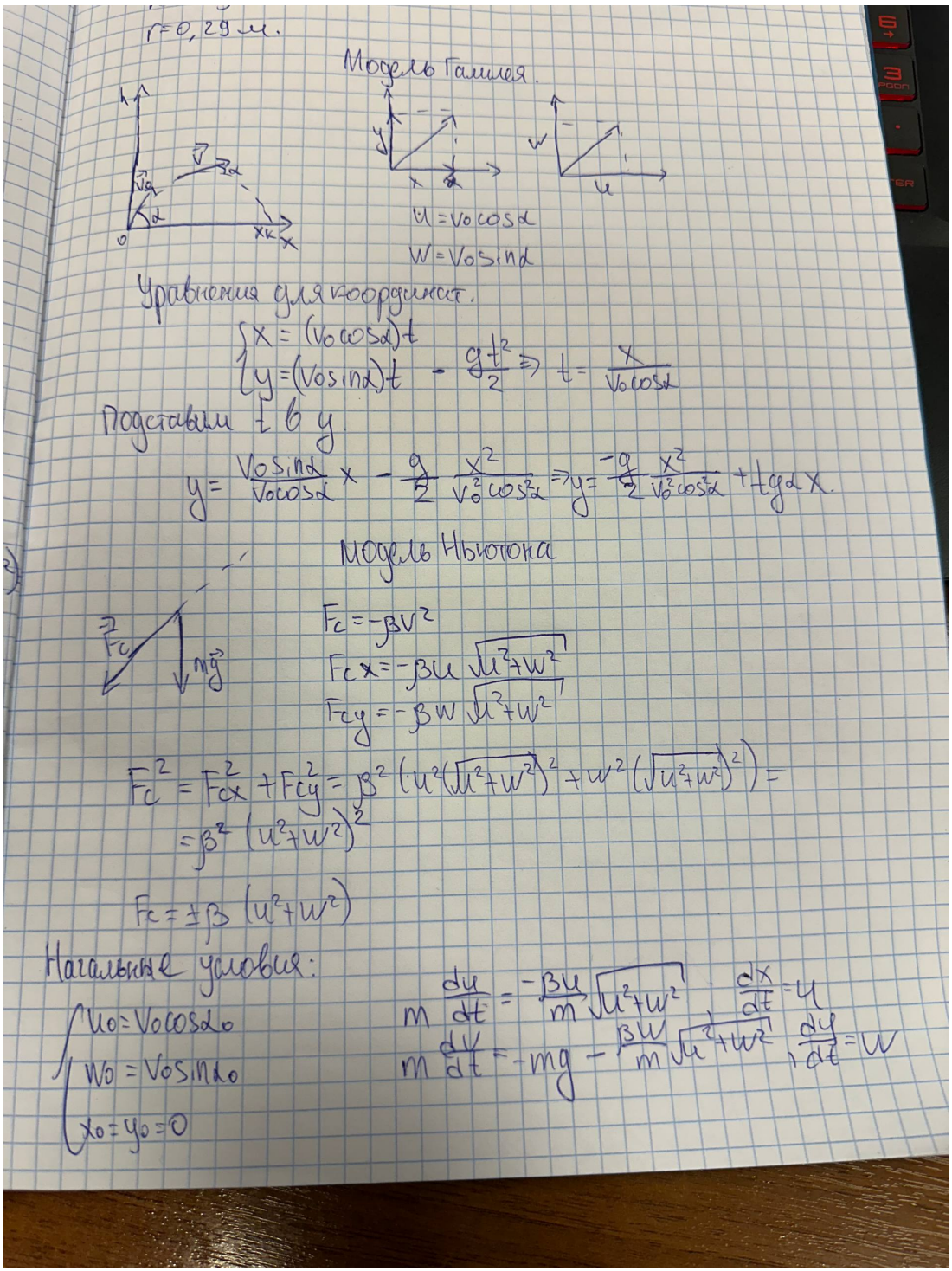

Рис. 1 — Схема построения математической модели

3 Разбиение на вычислительные задачи

3.1 Корректность вычислительной задачи

Вычислительная задача считается корректной, если решение существует, единственно и устойчиво к незначительным изменениям входных данных.

Данная задача имеет два решения, однако значение $x = 0, y = 0$ является начальными координатами, поэтому является вырожденным.

3.2 Решение системы ОДУ методом Рунге-Кутты

Система обыкновенных дифференциальных уравнений решается методом Рунге-Кутты 4-го порядка. Для уравнения $\frac{dy}{dt} = f(t, y)$ на шаге h по времени вычисляются четыре наклона и новое значение:

$$k_1 = f(t_n, y_n),$$

$$k_2 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1),$$

$$k_3 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2),$$

$$k_4 = f(t_n + h, y_n + hk_3),$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

На каждом шаге по текущему состоянию (x, y, u, w) вычисляются четыре вспомогательных вектора наклонов, комбинация которых даёт приближение на следующем шаге.

Линейная интерполяция на последнем шаге модели используется для уточнения точки падения: численная траектория переходит с положительной высоты на отрицательную между двумя узлами сетки. По двум последним точкам $(x_{\text{prev}}, y_{\text{prev}})$ и (x, y) строится линейная интерполяция по времени, находится момент и координата пересечения с $y = 0$, после чего в траекторию записывается эта точка приземления и возвращается уточнённая дальность полёта.

4 Анализ модели

График траекторий для варианта 29 представлен на рисунке 2.

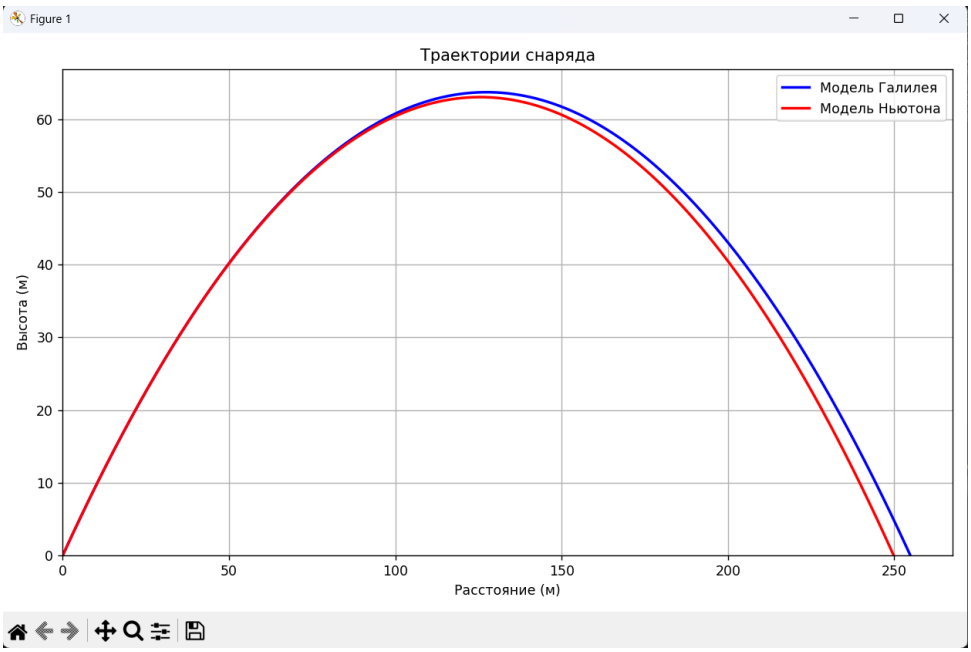

Рис. 2 — Траектории полёта: модель Галилея и модель Ньютона

5 Вывод

В ходе лабораторной работы были спроектированы математические модели Галилея и Ньютона для решения баллистической задачи для шара. Для решения задачи использовался метод Рунге-Кутты 4-го порядка. При анализе моделей было проведено сравнение результатов работы моделей. Ожидаемые результаты совпали с теми, что были получены в ходе тестирования.

6 Практическая реализация

Листинг 1. Исходный код программы.

```
import numpy as np
import matplotlib.pyplot as plt

def rk4_step(f, t, y, dt):
    k1 = f(t, y)
    k2 = f(t + dt/2, y + k1*dt/2)
    k3 = f(t + dt/2, y + k2*dt/2)
    k4 = f(t + dt, y + k3*dt)
    return y + dt*(k1 + 2*k2 + 2*k3 + k4)/6
```

```
def newton(v0, alpha, material_density=7000, air_density=1.225, C
    =0.15, dt=0.001, g=9.81):
    S = 0.03
    radius = (S/np.pi)**0.5
    m = material_density * (4/3) * np.pi * radius**3
    beta = C * air_density * S / 2.0
    u = v0 * np.cos(alpha)
    w = v0 * np.sin(alpha)
    x = 0.0
    y = 0.0
    t = 0.0
    max_t = max(4 * v0 * np.sin(alpha) / g, 30.0)
    x_prev, y_prev = x, y
    trajectory_x = [x]
    trajectory_y = [y]
    while t < max_t:
        state = np.array([x, y, u, w])
        def f(t0, s):
            ux, yy, uu, ww = s
            V = np.sqrt(uu*uu + ww*ww)
            if V < 1e-12:
                du = 0.0
                dw = -g
            else:
                du = -beta * uu * V / m
                dw = -g - beta * ww * V / m
            return np.array([uu, ww, du, dw])
        new = rk4_step(f, t, state, dt)
        x_prev, y_prev = x, y
        x, y, u, w = new
        trajectory_x.append(x)
        trajectory_y.append(y)
        t += dt
        if y < 0:
            if y == y_prev:
                return x, trajectory_x, trajectory_y
            frac = -y_prev / (y - y_prev)
            final_x = x_prev + frac*(x - x_prev)
            trajectory_x[-1] = final_x
            trajectory_y[-1] = 0.0
            return final_x, trajectory_x, trajectory_y
    return x, trajectory_x, trajectory_y

def galilei(v0, alpha, g=9.81):
```

```
    range_gal = v0*v0 * np.sin(2*alpha) / g
    t_max = 2 * v0 * np.sin(alpha) / g
    t_array = np.linspace(0, t_max, 1000)
    x_array = v0 * np.cos(alpha) * t_array
    y_array = v0 * np.sin(alpha) * t_array - 0.5 * g * t_array**2
    return range_gal, x_array.tolist(), y_array.tolist()

if __name__ == '__main__':
    alpha_deg = float(input('                                    : '))
    v0 = float(input('                                    : '))
    alpha = np.deg2rad(alpha_deg)

    range_gal, x_gal, y_gal = galilei(v0, alpha)
    range_newt, x_newt, y_newt = newton(v0, alpha)
    diff = range_gal - range_newt

    print(f'                                    (
                ): {range_gal:.6f}    ')
    print(f'                                    (
                ): {range_newt:.6f}    ')
    print(f'                : {diff:.6f}    ')

    plt.figure(figsize=(10, 6))
    plt.plot(x_gal, y_gal, 'b-', linewidth=2, label='
                ')
    plt.plot(x_newt, y_newt, 'r-', linewidth=2, label='
                ')
    plt.xlabel('                        (   )')
    plt.ylabel('                (   )')
    plt.title('                                    ')
    plt.grid(True)
    plt.legend()
    plt.xlim(left=0)
    plt.ylim(bottom=0)
    plt.tight_layout()
    plt.show()
```